# Software Engineer — Step 2

# DSA & OOP

**Roadmap objective:** Build strong algorithmic reasoning and object-oriented design skills
**Level:** Beginner → Intermediate
**Last reviewed:** September 2026

## How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

## Learning outcomes

- Arrays/lists
- Stacks/queues
- Linked lists
- Trees
- Hashing
- Sorting/searching
- Big-O
- Classes/objects
- Encapsulation
- Inheritance
- Polymorphism
- Composition

## Core concepts

1. DSA helps you choose appropriate representations and algorithms for a problem.

2. OOP is a modeling approach; use it when objects, responsibilities and boundaries make the design clearer.

3. Composition is often preferable to deep inheritance hierarchies because it reduces coupling.

4. Interview DSA practice should emphasize reasoning, complexity and edge cases—not only memorized solutions.

## Concept map

```
DSA & OOP
  |
  +--> Learn the concept
  |
  +--> Reproduce a small example
  |
  +--> Apply it in a feature
  |
  +--> Test failure/edge cases
  |
  +--> Document the decision
```

# Detailed study guide

## 1. Arrays/lists

Study arrays/lists through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

## 2. Stacks/queues

Study stacks/queues through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

## 3. Linked lists

Study linked lists through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

## 4. Trees

Study trees through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

## 5. Hashing

Study hashing through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

## 6. Sorting/searching

Study sorting/searching through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

# Worked example

## Simple class + composition

```
class Logger {
  info(message) {
    console.log("[INFO]", message);
  }
}

class OrderService {
  constructor(logger) {
    this.logger = logger;
  }
```

```
  createOrder(order) {
    this.logger.info("Creating order");
    return order;
  }
}
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

## Comparison table

| Concept | Meaning | Practical use |
|---|---|---|
| Inheritance | is-a relationship | Can couple hierarchies |
| Composition | has-a/uses relationship | Often more flexible |
| Encapsulation | Hide implementation | Protect invariants |

## Common mistakes

- Deep inheritance everywhere
- Classes with too many responsibilities
- Breaking encapsulation
- Ignoring complexity of data structures
- Memorizing patterns without understanding

## Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

# Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

### Lab 1

- Implement an array-based stack.
- Write down what you expected, what happened, and why.

### Lab 2

- Implement a queue.
- Write down what you expected, what happened, and why.

### Lab 3

- Solve one hashing/frequency problem.
- Write down what you expected, what happened, and why.

### Lab 4

- Implement a class with encapsulated state.
- Write down what you expected, what happened, and why.

### Lab 5

- Replace one inheritance relationship with composition.
- Write down what you expected, what happened, and why.

### Lab 6

- Write the complexity of each important operation.
- Write down what you expected, what happened, and why.

# Mini-project

Create a small library/order system using classes and data structures. Explain why each class exists, where composition is used, and the complexity of key operations.

### Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

# Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

## Array vs linked list?

- Answer structure: definition → example → trade-off/limitation → practical use.

## Stack vs queue?

- Answer structure: definition → example → trade-off/limitation → practical use.

## What is hashing?

- Answer structure: definition → example → trade-off/limitation → practical use.

## Encapsulation?

- Answer structure: definition → example → trade-off/limitation → practical use.

## Inheritance vs composition?

- Answer structure: definition → example → trade-off/limitation → practical use.

## How do you reason about complexity?

- Answer structure: definition → example → trade-off/limitation → practical use.

# Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

# Glossary

| Term | Your own definition | Example/use |
|---|---|---|
| Array | Write this in your own words. | Give one project example. |
| Stack | Write this in your own words. | Give one project example. |
| Queue | Write this in your own words. | Give one project example. |
| Tree | Write this in your own words. | Give one project example. |
| Hashing | Write this in your own words. | Give one project example. |
| OOP | Write this in your own words. | Give one project example. |
| Encapsulation | Write this in your own words. | Give one project example. |
| Inheritance | Write this in your own words. | Give one project example. |

| Term | Your own definition | Example/use |
|---|---|---|
| Composition | Write this in your own words. | Give one project example. |
| Polymorphism | Write this in your own words. | Give one project example. |

## Self-test

- Explain Arrays/lists without using its textbook definition.
- Show a small example of Stacks/queues.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

## Final checklist

- I can explain and demonstrate Arrays/lists.
- I can explain and demonstrate Stacks/queues.
- I can explain and demonstrate Linked lists.
- I can explain and demonstrate Trees.
- I can explain and demonstrate Hashing.
- I can explain and demonstrate Sorting/searching.
- I can explain and demonstrate Big-O.
- I can explain and demonstrate Classes/objects.
- I can explain and demonstrate Encapsulation.
- I can explain and demonstrate Inheritance.
- I can explain and demonstrate Polymorphism.
- I can explain and demonstrate Composition.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

## Recommended documentation

- https://leetcode.com/
- https://www.geeksforgeeks.org/

## Recommended videos

- Data Structures & Algorithms Course (Existing DB resource 117)
- Object-Oriented Programming Course (Existing DB resource 118)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.